



DSA Pattern Recognition Cheat Sheet

25 Patterns for Coding Interview Preparation

A quick-reference handbook to help you recognize the underlying pattern behind any DSA question — from the clues in its wording to the algorithm that solves it.

The goal is **not to memorize solutions**. Memorize the **trigger clues**.

Your mental process should become:

Question



Clues



Pattern



Algorithm

How to use this cheat sheet

1. **Read the trigger clues first.** Each of the 25 patterns lists the exact words and phrasings that usually signal it — train yourself to spot them inside a problem statement.
2. **Check the small bracketed notes** beside each clue — they explain *why* that phrase points to the pattern, not just *that* it does.
3. **Use the "Think:" line** as a one-sentence rule you can silently repeat while reading any new question.
4. **Run the 30-second recognition routine** (near the end of this sheet) before writing any code.
5. **Use the Ultimate Trigger Table** as a fast lookup once you're comfortable with all 25 patterns.

1 Sliding Window — Fixed Size

Trigger clues

- “subarray/substring of size **K**” (*contiguous portion with exactly K elements*)
- “every window of length K” (*process each K-sized window*)
- “maximum/minimum sum of K consecutive elements” (*fixed number of elements*)
- “average of every K elements” (*fixed-size window*)

Think: Contiguous + exact K → Fixed Sliding Window

Typical complexity: $O(n)$

2 Sliding Window — Variable Size

Trigger clues

- “longest subarray/substring” (*find the largest valid contiguous portion*)
- “shortest/minimum length subarray” (*find the smallest valid contiguous portion*)
- “at most K” (*maximum allowed quantity is K*)
- “sum $\geq$ target” (*window must reach a minimum sum*)
- “sum $\leq$ target” (*window must stay within a limit*)
- “contains at most K distinct elements” (*limit the number of unique values*)

Think: Contiguous + condition + longest/shortest → Variable Sliding Window

Examples

- Longest Substring Without Repeating Characters
- Minimum Size Subarray Sum
- Fruit Into Baskets

3 Two Pointers

Trigger clues

- Array is **sorted** (*ordering can help eliminate possibilities*)
- “find a pair” (*usually two elements must satisfy a condition*)
- “find a triplet” (*often reduce the problem using two pointers*)
- “two ends of an array matter” (*left and right boundaries are useful*)
- Need to move `left` and `right` (*two indices scan the data*)

Think: Sorted + pair/triplet → Two Pointers

```
[-4, -1, 0, 2, 5, 7]
```

4 Fast & Slow Pointers

Trigger clues

- Linked list cycle (*detect whether nodes form a loop*)
- Detect cycle (*determine whether traversal repeats*)
- Find middle of linked list (*use different pointer speeds*)
- Find cycle starting point (*locate where the loop begins*)
- “Move one pointer twice as fast” (*classic slow/fast technique*)

Think: Linked list + cycle/middle → Fast & Slow

```
slow → 1 step  
fast → 2 steps
```

5 HashMap / HashSet

🔥 Trigger clues

- Need **frequency** (*count how often values occur*)
- Need to know whether something appeared before (*fast existence check*)
- Duplicate detection (*check repeated values*)
- "Have we seen this?" (*track previously encountered values*)
- Count occurrences (*frequency calculation*)
- Need `O(1)` lookup (*constant-time average lookup*)

Think: Need fast lookup/count → Hashing

Examples

- Two Sum
- Contains Duplicate
- Frequency Counting
- Anagram

6 Prefix Sum

🔥 Trigger clues

- Repeated **range sum** (*calculate many subarray/range sums*)
- "sum from L to R" (*query a specific range*)
- Subarray sum (*calculate sums of contiguous portions*)
- Need cumulative information (*store running totals*)
- Many queries on ranges (*answer repeated range calculations efficiently*)

Think: Repeated range calculations → Prefix Sum

```
prefix[i] = prefix[i-1] + arr[i]
```

7 Prefix Sum + HashMap

Trigger clues

- "Count subarrays with sum K" (*count contiguous ranges whose sum equals K*)
- "Does a subarray have sum K?" (*check whether such a range exists*)
- Negative numbers present (*normal sliding window may fail*)
- Need number of subarrays satisfying a sum condition (*count valid ranges*)

Think: Subarray sum + arbitrary/negative numbers → Prefix Sum + HashMap

```
prefixSum - K
```

8 Binary Search

Trigger clues

- Array is sorted (*ordered data allows elimination of half the search space*)
- Search efficiently (*avoid linear scanning*)
- Find first/last occurrence (*boundary search*)
- Lower bound / upper bound (*find insertion boundaries*)
- Find minimum/maximum satisfying a condition (*search for a valid boundary*)

Think: Sorted/search space → Binary Search

Typical complexity: $O(\log n)$

9

Binary Search on Answer

 Very important pattern

Trigger clues

- “minimum possible...” *(find the smallest feasible answer)*
- “maximum possible...” *(find the largest feasible answer)*
- “smallest X such that...” *(search for first valid X)*
- “largest X such that...” *(search for last valid X)*
- “Can we do it in X?” *(turn optimization into yes/no checking)*
- Answer lies within a range *(there is a searchable answer space)*
- You can write a `check(mid)` function *(test whether a candidate answer works)*

Think: Optimization + Yes/No feasibility → Binary Search on Answer

10

Stack — Monotonic Stack

Trigger clues

- Next greater element *(first larger element to the right)*
- Next smaller element *(first smaller element to the right)*
- Previous greater/smaller *(nearest qualifying element on the left)*
- Nearest greater/smaller *(find the closest element satisfying a comparison)*
- Histogram *(bars often require nearest smaller boundaries)*
- “First element greater than...” *(find the nearest larger element)*

Think: Nearest greater/smaller → Monotonic Stack

Examples

- Next Greater Element
- Daily Temperatures
- Largest Rectangle in Histogram

11 Stack — Parentheses / Expression

Trigger clues

- Balanced parentheses (*match opening and closing brackets*)
- Valid brackets (*check correct nesting/order*)
- Nested expressions (*last opened structure closes first*)
- Undo-like behavior (*most recent operation is handled first*)
- Last opened → first closed (*LIFO behavior*)

Think: Nested / matching structure → Stack

```
{ [ ( ) ] }
```

12 Heap / Priority Queue

Trigger clues

- Top K (*only the K most important elements are needed*)
- K largest/smallest (*maintain K extreme elements*)
- Repeatedly get minimum (*always need the smallest current value*)
- Repeatedly get maximum (*always need the largest current value*)
- Maintain current smallest/largest (*dynamic min/max tracking*)
- Scheduling based on priority (*process the highest-priority item first*)

Think: Repeated min/max → Heap

Examples

- Kth Largest
- Top K Frequent
- Merge K Sorted Lists
- Meeting Rooms

13 Greedy

Trigger clues

- Make the **best choice now** (*choose the locally optimal option*)
- Maximize/minimize something (*optimization objective*)
- Choice doesn't need to be revisited (*decision can be permanently committed*)
- "Earliest finishing" (*choose the activity that leaves maximum future space*)
- "Maximum number of activities" (*select the most compatible activities*)
- "Minimum number of..." (*sometimes solved through optimal local choices*)
- Sorting enables local decisions (*ordering reveals the greedy choice*)

Think: Local optimal choice → possibly Greedy

 **Do not assume greedy only because the question asks for a maximum/minimum.**

14 Interval Pattern

Trigger clues

- `[start, end]` (*each item represents a range*)
- Meetings (*time intervals may overlap*)
- Events (*events occupy time ranges*)
- Jobs (*jobs have start/end times*)
- Overlapping intervals (*ranges intersect*)
- Merge intervals (*combine overlapping ranges*)
- Insert interval (*add a new range while preserving order*)
- Scheduling (*choose or organize time intervals*)

Think: Start/end ranges → Sort by start/end

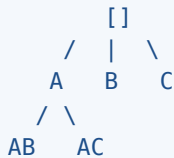
```
sort(intervals.begin(), intervals.end());
```


15 Backtracking

🔥 Trigger clues

- “Generate all...” (*need every valid possibility*)
- “Find all possible...” (*explore multiple choices*)
- Combinations (*choose elements without caring about order*)
- Permutations (*arrange elements in different orders*)
- Subsets (*choose or skip each element*)
- Choices at every step (*multiple decisions create a search tree*)
- Need to explore possibilities (*try a choice, then undo it*)

Think: All possibilities → Backtracking



16 Bit Manipulation

🔥 Trigger clues

- XOR (*use properties of binary bits*)
- Set/unset bits (*modify individual bits*)
- Odd/even bit behavior (*inspect binary representation*)
- Powers of 2 (*binary numbers with one set bit*)
- “Only one number appears once” (*XOR can cancel duplicates*)
- “Every number appears twice except...” (*XOR pairs cancel to zero*)
- Binary representation (*problem directly involves bits*)

Think: Binary properties → Bits

$$x \wedge x = 0$$
$$x \wedge 0 = x$$

17 Recursion / Tree DFS

Trigger clues

- Binary tree (*naturally represented recursively*)
- Subtree (*solve the same problem on a smaller tree*)
- Height/depth (*combine information from children*)
- Root-to-leaf (*follow one recursive path*)
- Path (*combine information along tree paths*)
- Left/right child (*process both subtrees*)
- "For every node..." (*perform the same operation recursively*)

Think: Tree → Ask the left and right subtrees

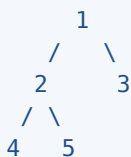
```
solve(node->left);  
solve(node->right);
```

18 Tree BFS / Level Order

Trigger clues

- Level by level (*process nodes based on depth*)
- Level order (*visit each tree level together*)
- Distance from root (*depth can be measured using BFS*)
- Minimum number of levels (*find the closest level*)
- Zigzag traversal (*alternate direction at each level*)
- Nodes at distance K (*process nodes by distance*)

Think: Tree + levels → BFS / Queue



Level 0 → 1
Level 1 → 2, 3
Level 2 → 4, 5

19 BST Pattern

Trigger clues

- Binary **Search Tree** (*left < root < right ordering*)
- Left < root < right (*BST ordering property*)
- Sorted order (*inorder traversal produces sorted values*)
- Find predecessor/successor (*use BST ordering*)
- Kth smallest (*inorder gives sorted sequence*)
- Search/insert/delete (*BST property reduces search area*)

Think: BST → Exploit ordering

 **Inorder traversal of BST = sorted order**

20 Graph BFS / DFS

Trigger clues

- Connected components (*find separate connected groups*)
- Islands (*grid cells form connected components*)
- Reachability (*determine whether one node can reach another*)
- "Can we reach..." (*graph traversal problem*)
- Visit all connected nodes (*explore one component*)
- Grid with neighboring cells (*treat cells as graph nodes*)

Think: Connectivity → DFS/BFS

Examples

- Number of Islands
- Flood Fill
- Connected Components

21 Graph BFS — Shortest Path

Trigger clues

- **Shortest path** (*minimum distance between nodes*)
- Minimum number of steps (*each move has equal cost*)
- Every edge has equal weight (*BFS levels represent distance*)
- Grid movement (*each move usually costs one step*)
- “Fewest moves” (*minimum number of transitions*)

Think: Unweighted graph + shortest path → BFS

```
distance 0
  ↓
distance 1
  ↓
distance 2
  ↓
distance 3
```

22 Dijkstra

Trigger clues

- Shortest path (*find minimum total path cost*)
- **Weighted graph** (*edges have costs/weights*)
- Edge weights are **non-negative** (*Dijkstra requires non-negative weights*)
- Need minimum distance from source (*single-source shortest path*)

Think: Weighted + non-negative → Dijkstra

```
priority_queue<
    pair<int,int>,
    vector<pair<int,int>>,
    greater<pair<int,int>>
> pq;
```

23 Topological Sort

Trigger clues

- Prerequisite (*one task must happen before another*)
- Dependency (*tasks depend on other tasks*)
- Course schedule (*courses have prerequisite relationships*)
- Build order (*dependencies determine execution order*)
- Task ordering (*find a valid sequence of tasks*)
- "A must happen before B" (*directed dependency*)
- Directed graph (*topological ordering applies to DAGs*)

Think: Dependency/order → Topological Sort

Examples

- Kahn's Algorithm → BFS + indegree
- DFS

24 Dynamic Programming

Trigger clues

- "Maximum/minimum number of..." (*optimize a result*)
- "Number of ways..." (*count different valid choices*)
- "Can we achieve..." (*determine whether a state is reachable*)
- Repeated subproblems (*same smaller problem appears multiple times*)
- Choices at every position (*different decisions lead to different states*)
- Previous decisions affect future (*current answer depends on previous state*)
- Brute force recursion has overlapping states (*memoization can remove repeated work*)

Think: Choices + overlapping subproblems → DP

```
What is my state?  
What choices do I have?  
What is the transition?
```

25 Union Find / DSU

Trigger clues

- Dynamic connectivity (*connections change over time*)
- Merge groups (*combine two components*)
- Are A and B connected? (*check whether two nodes belong to the same component*)
- Number of connected components (*track separate groups*)
- Connecting nodes/edges (*adding relationships between groups*)
- Detect cycle in undirected graph (*cycle detection using components*)
- Kruskal's algorithm (*MST algorithm based on DSU*)

Think: Repeatedly merge/check groups → DSU

```
find(x)
union(a,b)
```

Quick Reminders

- **Binary Search on Answer** is one of the highest-leverage patterns — whenever you can write a `check(mid)` feasibility function, the problem is very likely searchable.
- **Greedy** should never be assumed just because a question asks for a maximum or minimum — verify the local-choice logic actually holds before committing to it.
- **BST** problems almost always reduce to one fact: inorder traversal of a BST produces sorted order.
- When brute-force recursion has **overlapping subproblems**, that overlap — not the recursion itself — is the real signal for **Dynamic Programming**.

The Ultimate Trigger Table

A fast lookup once you already know the 25 patterns — skim the left column, jump to the right.

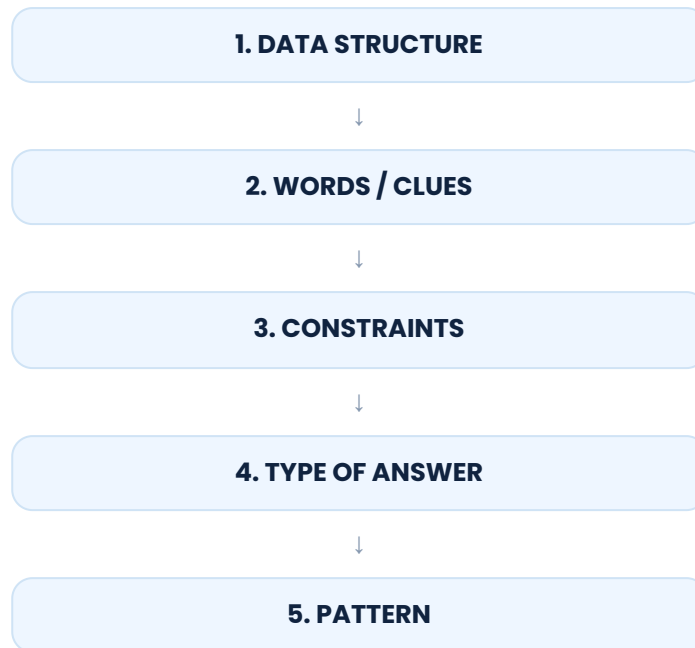
If question says...	Suspect...
Subarray + size K	Fixed Sliding Window
Longest/shortest subarray	Variable Sliding Window
Sorted + pair	Two Pointers
Linked list + cycle	Fast/Slow
Frequency/count/seen	HashMap
Range sum	Prefix Sum
Subarray sum K	Prefix Sum + HashMap
Sorted/search space	Binary Search
Minimum possible X	Binary Search on Answer
Next greater/smaller	Monotonic Stack
Balanced brackets	Stack
Top K	Heap
Repeated min/max	Heap

If question says...	Suspect...
Start/end ranges	Intervals
All possible	Backtracking
Binary/XOR	Bit Manipulation
Tree path/height	DFS
Tree levels	BFS
BST	Inorder / BST properties
Connected components	DFS/BFS/DSU
Shortest unweighted path	BFS
Shortest weighted path	Dijkstra
Prerequisites/dependencies	Topological Sort
Choices + overlapping states	DP
Merge/check groups	DSU
Local optimal choice	Greedy

The REAL Trick

Don't try to recognize the **algorithm** first.

Recognize these **5 things**, in order:



30-Second Pattern Recognition Routine

- 1 What is the input structure? Array / String / Linked List / Tree / Graph?
- 2 What are the important words? longest / shortest / count / all / minimum / maximum?
- 3 Is it contiguous? Subarray / substring?
- 4 Is it sorted?
- 5 What are the constraints? $O(n)$, $O(n \log n)$, $O(n^2)$?
- 6 What operation is repeated? lookup / min / max / nearest / connectivity?
- 7 What pattern does this remind me of?

Pattern: _____

Reason: _____



The Goal

If you practice this deliberately for **50–100 problems**, you'll notice a major change:

Instead of seeing **100 different DSA questions**, you'll start seeing roughly **20–30 recurring problem shapes**.

That's the actual skill you're building:

Pattern Recognition.

